

Week 4: SVMs and Backpropagation

<http://mlvu.github.io>

February 27, 2023

1 Support vector machines

We will practice a little with the maximum margin loss/support vector machine to ensure that you understand the details.

question 1: Jack says that support vector machines are a way to learn non-linear functions, that's what they're famous for. They use the kernel trick to do this. Jill says that Jack is wrong: support vector machines use a *linear* classifier trained by the maximum-margin loss function. Who, if anybody, is right?

Both are a little right and a little wrong. The simplest form of support vector machine is, as Jill says, a linear classifier trained to minimize a particular loss. However this specific loss means that you can rewrite the minimization problem to a form where you can apply something known as the kernel trick. This allows you to learn nonlinear functions at the cost of fitting a linear classifier. This is what Jack is referring to.

Support Vector machines were popular for a long time because of this kernel trick, so usually when you will see them used, it will be as nonlinear models. If the linear version is used, it will be referred to as a *linear* support vector machine.

In the linear case, the (soft margin) SVM finds a hyperplane, expressed as $\mathbf{w}^T \mathbf{x} + \mathbf{b}$, by solving the following optimization problem:

$$\begin{aligned} &\text{minimize } \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i p_i \\ &\text{such that } y_i (\mathbf{w}^T \mathbf{x}_i + \mathbf{b}) \geq 1 - p_i \\ &\text{and } p_i \geq 0 \end{aligned}$$

question 2: What does i index? How many terms does the sum have, and how many constraints are there? It iterates over the data. There are as many terms as there are instances in the data. There are two constraints per instance.

question 3: What is the value of y_i in this expression? What is its function? y_i is 1 for positive examples, and -1 for negative examples. It allows us to rewrite the constraints

$$\begin{aligned} \mathbf{w}^T \mathbf{x}_i + b &\geq 1 \text{ if } \mathbf{x}_i \text{ is positive} \\ \mathbf{w}^T \mathbf{x}_i + b &\leq -1 \text{ if } \mathbf{x}_i \text{ is negative} \end{aligned}$$

to a single constraint.

After training, the class of an instance $\mathbf{x}$ is predicted based on the sign of $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$. If $f(\mathbf{x})$ is negative, then the class is **negative**, and if it is positive, then the class is **positive**.¹

question 4: After training a linear SVM, we get $\mathbf{w} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}$ and $b = -1$. Is

$\mathbf{x}_i = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$ with $y_i = 1$ is a support vector?
Yes, it is.

$$\mathbf{w}^T \mathbf{x}_i + b = \begin{pmatrix} 5 \\ 3 \end{pmatrix}^T \begin{pmatrix} 1 \\ -1 \end{pmatrix} - 1 =$$

so the expression $\mathbf{w}^T \mathbf{x}_i + b = y_i$ and $\mathbf{x}_i$ is a support vector.

question 5: Does this mean that we expect the resulting classifier to satisfy $f(\mathbf{x}_i) = y_i$ for every instance, as in a regression problem?

No. We expect this only for specific instances. See the next question.

question 6: Having resolved their previous disagreement Jack and Jill are now a little confused about what the “support vectors” are. They agree that they are a subset of the instances in the data. Jack thinks that they are instances for which $f(\mathbf{x}_i)$ is exactly equal to y_i . Jill thinks that they are the instances that are exactly on the margin defined by the classifier. Who, if anybody, is right?

¹SVMs are only defined for binary classification problems. In binary classification, we commonly call one class **positive** and the other **negative**. For multiclass problems, it's common to train one SVM per class to distinguish the class from the rest, and to combine these into a single classifier.

Both are right. The margin of the classifier is defined as the region between the hyperplane where $f(\mathbf{x}) = -1$ and the hyperplane where $f(\mathbf{x}) = 1$. See [these slides](#) for an illustration. If an instance is on the correct side of the margin, that means that for a **negative** point $f(\mathbf{x})$ is exactly -1 and for a **positive** point $f(\mathbf{x})$ is exactly 1 .

We can summarize this by saying that a data point is a support vector if it satisfies the following conditions:

$$\mathbf{w}^T \mathbf{x}_i + \mathbf{b} = y_i$$

If this condition is met, then point $\mathbf{x}_i$ lies exactly on one of the support vectors. So Jill is correct as well.

For the remainder, we will use the following classifier:

$$\mathbf{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, \mathbf{b} = 1$$

question 7: What classes are assigned to the points $\begin{pmatrix} 2 \\ 1 \end{pmatrix}$, $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$ and $\begin{pmatrix} 2 \\ 2 \end{pmatrix}$.

point 1:

$$\begin{pmatrix} 1 \\ -2 \end{pmatrix}^T \begin{pmatrix} 2 \\ 1 \end{pmatrix} + 1 = 1 \cdot 2 - 2 \cdot 1 + 1 = 1$$

The result is positive so the class is **positive**.

point 2:

$$\begin{pmatrix} 1 \\ -2 \end{pmatrix}^T \begin{pmatrix} 1 \\ 2 \end{pmatrix} + 1 = 1 \cdot 1 - 2 \cdot 2 + 1 = -2$$

The result is negative so the class is **negative**.

point 3:

$$\begin{pmatrix} 1 \\ -2 \end{pmatrix}^T \begin{pmatrix} 2 \\ 2 \end{pmatrix} + 1 = 1 \cdot 2 - 2 \cdot 2 + 1 = -1$$

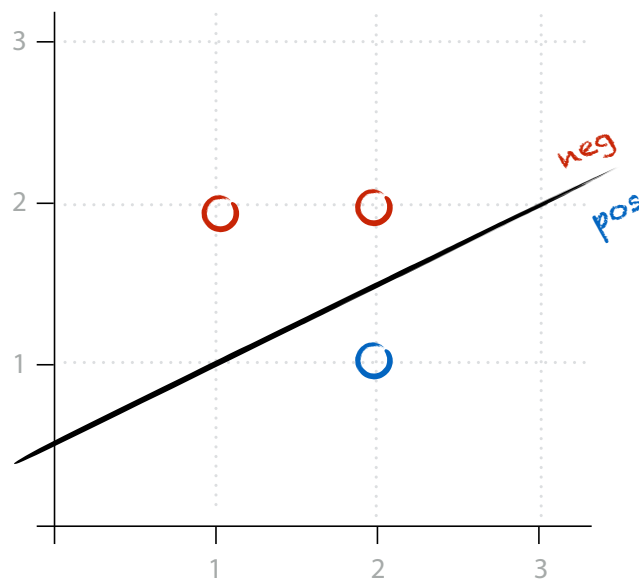
The result is negative so the class is **negative**.

question 8: Draw the decision boundary in feature space, and label the points from the previous question. Label which side of the decision boundary is **positive** and which is **negative**. *Tip: what is the value of $\mathbf{w}^T \mathbf{x} + \mathbf{b}$ when $\mathbf{x}$ is exactly on the decision boundary? Write this as a scalar equation, and rewrite to get the familiar representation of a line.*

At the decision boundary, $\mathbf{w}^T \mathbf{x} + b = 0$. For our classifier that gives us

$$\begin{aligned} 1x_1 - 2x_2 + 1 &= 0 \\ -2x_2 &= -1x_1 - 1 \\ x_2 &= \frac{1}{2}x_1 + \frac{1}{2} \end{aligned}$$

This is the line that crosses the x_2 axis at $\frac{1}{2}$ and has a slope of $\frac{1}{2}$



question 9: Which of the following pairs are support vectors?

$$\mathbf{x}_1 = \begin{pmatrix} 1 \\ 3/2 \end{pmatrix}, y_1 = 1$$

$$\mathbf{x}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, y_2 = -1$$

$$\mathbf{x}_3 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, y_3 = 1$$

$$\mathbf{x}_4 = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}, y_4 = 1$$

A pair $\mathbf{x}, y$ is a support vector if $\mathbf{w}^T \mathbf{x} + b = y$. This is the case for points $\mathbf{x}_2$ and $\mathbf{x}_4$.

question 10: Draw the margins of the classifier in the previous diagram, together with the points from the previous question. *Tip: the edges of the margins are defined by an equation, very similar to that of the decision boundary.*

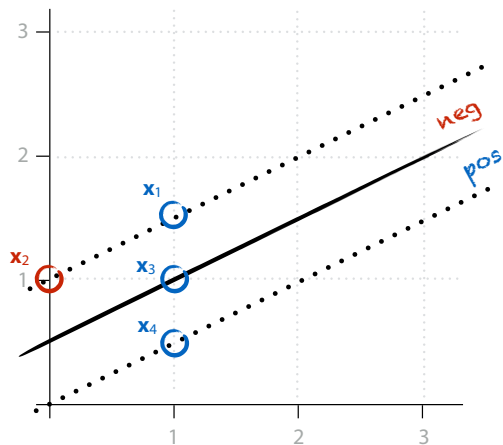
Let's start with the positive edge of the margin. This is the line (or hyperplane) where $\mathbf{w}^T \mathbf{x} + b = 1$. For our classifier that means that

$$\begin{aligned} 1x_1 - 2x_2 + 1 &= 1 \\ -2x_2 &= -1x_1 \\ x_2 &= \frac{1}{2}x_1. \end{aligned}$$

For the negative edge, we have $\mathbf{w}^T \mathbf{x} + b = -1$, so

$$\begin{aligned} 1x_1 - 2x_2 + 1 &= -1 \\ -2x_2 &= -1x_1 - 2 \\ x_2 &= \frac{1}{2}x_1 + 1. \end{aligned}$$

These are the lines with slope $\frac{1}{2}$ with pass through the x_2 axis at 0 and 1 respectively.



Note that even though x_1 is on the margin, its label is $y = 1$, so it's on the *wrong* margin. If this point were in our dataset, we would have a violation of the requirement that all points are on the right side of the margin. For a hard margin SVM, that means that this classifier is invalid, for a soft margin SVM, it would mean that we had to pay a penalty.

question 11: To finish up with, here's a bit of puzzle. You don't have to be able to do this for the exam, but trying to solve it should help you to understand the finer details of how the parameters of the SVM determine the margin and the decision boundary.

The slides say that there are infinitely many classifiers with the same decision boundary, but that they all have different margins. Find a classifier that has the same decision boundary as the one we've been using so far, but whose margin is twice as wide.

There are two ways to solve this. The first is by drawing the double sized margin in our picture. If the decision boundary is the same, the new margins should be parallel to the old, but twice as large. This means that they should cross the x_2 axis at $-\frac{1}{2}$ and $\frac{3}{2}$, with the slope staying $\frac{1}{2}$.

That is, from the parameters of our classifier, we should get the following equations for the edges of our margin:

$$x_2 = \frac{1}{2}x_1 - \frac{1}{2} \qquad x_2 = \frac{1}{2}x_1 + \frac{3}{2} \qquad (1)$$

Now, for some parameters w_1 , w_2 and b , we will derive the resulting margins as

$$\begin{aligned} w_1x_1 + w_2x_2 + b &= 1 & w_1x_1 + w_2x_2 + b &= -1 \\ w_2x_2 &= 1 - w_1x_1 - b & w_2x_2 &= -1 - w_1x_1 - b \\ x_2 &= \frac{1}{w_2} - \frac{w_1}{w_2}x_1 - \frac{b}{w_2} & x_2 &= -\frac{1}{w_2} - \frac{w_1}{w_2}x_1 - \frac{b}{w_2} \end{aligned}$$

This is just what we did before to work out the boundaries, but now with symbols.

Now, we just need to figure out which values of w_1 , w_2 and b will make these equations match (1). Matching up the coefficients, we get:

$$\begin{aligned} -\frac{w_1}{w_2} &= \frac{1}{2} \\ w_1 &= -2w_2 \end{aligned}$$

and

$$\frac{1 - \mathbf{b}}{\mathbf{w}_2} = -\frac{1}{2} \qquad \frac{1 + \mathbf{b}}{\mathbf{w}_2} = -\frac{3}{2}.$$

If we assume that $\mathbf{w}_2$ is negative, then $1 + \mathbf{b}$ needs to be three times as big as $1 - \mathbf{b}$. We get this for $\mathbf{b} = \frac{1}{2}$. If this is correct, then for the above to work, we need $\mathbf{w}_2 = -1$, which immediately tells us that $\mathbf{w}_1 = \frac{1}{2}$.

With these, we can repeat the procedure from question 8 to confirm that they yield the correct margin.

The second approach is to use something that was worked out in the slides. The width of the margin is $2/\|\mathbf{w}\|$. That means that to make the margin twice as big, we need to make the norm of $\|\mathbf{w}\|$ twice as small.

Moreover, to keep the decision boundary the same we need to keep $\mathbf{w}$ pointing in the same direction (since it's the vector that is orthogonal to the decision boundary).

We achieve both by halving both its elements, giving us $\mathbf{w} = \begin{pmatrix} 1/2 \\ -1 \end{pmatrix}$. All that's left is to figure out what $\mathbf{b}$ should be. Since the decision boundary is the same as before, it should cross the x_2 axis at the same point as before, which gives us $\frac{1}{2} \cdot 0 - 1 \cdot \frac{1}{2} + \mathbf{b} = 0$, and therefore $\mathbf{b} = \frac{1}{2}$.

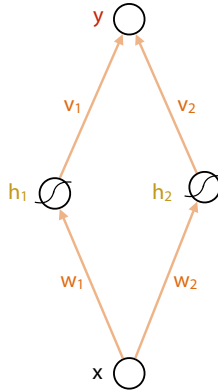
2 Backpropagation

We will practice the backpropagation algorithm as described in the slides. We will use a neural network defined by the following function:

$$\begin{aligned} \mathbf{y} &= \mathbf{v}_1 \mathbf{h}_1 + \mathbf{v}_2 \mathbf{h}_2 \\ \mathbf{h}_1 &= \sigma(\mathbf{k}_1) \\ \mathbf{h}_2 &= \sigma(\mathbf{k}_2) \\ \mathbf{k}_1 &= \mathbf{w}_1 \mathbf{x} \\ \mathbf{k}_2 &= \mathbf{w}_2 \mathbf{x} \end{aligned}$$

Where σ represents the logistic sigmoid. The network has a single input node $\mathbf{x}$ and a single output node $\mathbf{y}$. The weights are $\mathbf{w}_1$, $\mathbf{w}_2$, $\mathbf{v}_1$ and $\mathbf{v}_2$. We've left out bias nodes to keep things simple.

question 12: Draw this network.



We will use stochastic gradient descent to train this network. That means we define the loss function for a single instance. We will use basic least-squares loss to train this network for regression. Our loss function for instance x is

$$\text{loss}_x(w_1, w_2, v_1, v_2) = \frac{1}{2} (y - t)^2$$

where t is the target value provided by the dataset, and y is the output of the network.

We see each line in the definition above as a module, with some inputs and some outputs. Using the chain rule, we will express the derivative with respect to weight w_1 . We will use only *local derivatives*, expressing the derivative for each module with respect to its inputs, but not working out the derivative beyond that.

question 13: Fill in the gaps:

$$\begin{aligned} \frac{\partial \text{loss}}{\partial w_1} &= \frac{\partial \text{loss}}{\partial y} \frac{\partial y}{\partial h_1} \frac{\partial h_1}{\partial k_1} \frac{\partial k_1}{\partial w_1} \\ &= (y - t) \times v_1 \times \sigma(k_1)(1 - \sigma(k_1)) \times x \end{aligned} \quad (2)$$

question 14: The network has a diamond shape, just as shown in the slide when explaining the multivariate chain rule (in the *Deep Learning 1* lecture). However, in this case, we don't need the multivariate chain rule. Why not? In what kind of situation would the multivariate chain rule be required?

The multivariate chain rule is needed when the output depends on one of the variables for which we are taking the derivative (like w_1) along multiple paths in the computation graph. In this case we have a diamond because y

depends on x along two paths, but we never take the derivative with respect to x (the input), only with respect to the weights.

In a way, we *are* using the multivariate chain rule, if we write

$$\frac{\partial l}{\partial w_1} = \frac{\partial v_1 h_1}{\partial w_1} + \frac{\partial v_2 h_2}{\partial w_1}$$

(because the sum rule is an instance of the multivariate chain rule) but the second term becomes zero because the numerator is constant with respect to w_1 .

The multivariate chain rule comes into effect when the output depends on a *weight* along multiple paths in the computation graph. This happens, for instance, when we compute the loss function over a batch of multiple data points.

We could take the formulation above and fill in the symbolic expressions for y , k_1 , etc, and come up with a general symbolic formula for the derivative for all inputs. However, that is usually expensive to do for large neural nets. Instead, we leave it as is, and fill in the *numeric* values for these variables for a specific input and for specific weights.

question 15: Assume that the input is $x = 1$, with target output $t = \frac{1}{2}$ and that **all weights** are set at 1. Do a *forward pass*: compute the loss and all intermediate values k_1 , k_2 , h_1 , h_2 and y .

To simplify the calculation, you can use the approximation $\sigma(1) = \frac{3}{4}$.

$$\begin{aligned} k_1 &= 1 \\ k_2 &= 1 \\ h_1 &= 3/4 \\ h_2 &= 3/4 \\ y &= 3/2 \\ \text{loss} &= \frac{1}{2} \end{aligned}$$

Now we do the *backward pass*. Fill these intermediate values in to the loss function decomposed by the chain rule from line 2, and compute the derivative with respect to w_1 for these inputs and weights.

$$\frac{\partial \text{loss}}{\partial w_1} = 1 \times 1 \times \frac{3}{4} \times \frac{1}{4} \times 1 = \frac{3}{16}$$

3 Bonus: Normalization

This section was part of an earlier version of the homework. It doesn't correspond to an application question, so you can skip this if you like, but if you want to practice the basis transformations from the normalization video, you can use these exercises.

In this section we'll look at the theory behind whitening. The idea of a *basis transformation* is important to understand, so we'll start there.

Basis systems The *standard basis* (in 2 dimensions) is just the vectors $b_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $b_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. When a point has coordinates (x, y) , that just means that we can get to the point by adding x times the b_1 vector to y times the b_2 vector.

For the standard basis, we just end up at the point $\begin{pmatrix} x \\ y \end{pmatrix}$. But we can also use a nonstandard basis. If a point p has coordinates (x, y) in the coordinate system with basis vectors b_1, b_2 , it means that p has coordinates $xb_1 + yb_2$ in the standard basis system.

question 16: Convert the following points to a representation in the standard basis system.

1. The point $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ expressed in the coordinate system with basis vectors $b_1 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, b_2 = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$. $x = 1, y = 1$, so we get to the standard basis point by computing $1 \cdot b_1 + 1 \cdot b_2 = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$
2. The point $\begin{pmatrix} 5 \\ 3 \end{pmatrix}$ expressed in the coordinate system with basis vectors $b_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, b_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$. $\begin{pmatrix} 3 \\ 5 \end{pmatrix}$
3. The point $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$ expressed in the coordinate system with basis vectors $b_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, b_2 = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$. $\begin{pmatrix} 7 \\ 10 \end{pmatrix}$

question 17: A basis system is **orthonormal** if two conditions hold:

1. The basis vectors all have norm (i.e. length) 1. (normality)
2. The angles between any two basis vectors are 90 deg. (orthogonality)

Consider the following three bases:

$$\mathbf{A} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \quad \mathbf{B} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \quad \mathbf{C} = \sqrt{1/2} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

The columns of each matrix are the basis vectors (it may help to draw them in standard coordinates). Which of these have orthogonal basis vectors? Which one is orthonormal? The columns of **A** are neither normal nor orthogonal. Matrix **B** has orthogonal columns, but they are not unit vectors (vectors of length one). Matrix **C** represents an orthonormal basis.

question 18: Why is it useful to have an orthonormal basis? Matrices representing an orthonormal basis are *orthogonal*. This means that $\mathbf{C}^{-1} = \mathbf{C}^T$. This means that we can easily transform from and to the basis without having to compute an expensive matrix inverse.

Covariance and normal distributions PCA is closely related to the normal distribution. We can think of PCA as a basis transformation that makes it look like the data was sampled from a standard multivariate normal distribution: it makes the variance 1 in every dimension, and the covariance 0 for every pair of dimensions.

question 19: Here is some data.

x_1	x_2
0	2
2	0
0	2
2	0

Compute the sample covariance.

We first compute the sample mean: $\mathbf{m} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$.

Subtracting the sample mean, and arranging the data in a matrix (with instances as columns), we get

$$\mathbf{X} = \begin{pmatrix} -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \end{pmatrix}.$$

The sample covariance is

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X} \mathbf{X}^T = \frac{1}{3} \begin{pmatrix} 4 & -4 \\ -4 & 4 \end{pmatrix}$$

Let $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{t}$ be an *affine* transformation (a linear transformation $\mathbf{A}$ followed by a translation $\mathbf{t}$). We draw $\mathbf{x}$ from a standard normal distribution

$$\mathbf{N}(\mathbf{0}, \mathbf{I}) = \mathbf{N}\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\right)$$

and compute $\mathbf{y}$. This is equivalent to sampling from a normal distribution with mean $\boldsymbol{\mu} = \mathbf{t}$ and covariance $\boldsymbol{\Sigma} = \mathbf{A}\mathbf{A}^T$.

question 20: Let

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \mathbf{t} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Sampling $\mathbf{x}$ from $\mathbf{N}(\mathbf{0}, \mathbf{I})$ is easy: we just sample x_1 and x_2 independently from a *univariate* standard normal distributions. Here are some **standard normally distributed numbers**:

$$0.5, 1.1, 0.1, 0.9, -0.2, 1.3$$

Turn these into three samples from $\mathbf{N}(\mathbf{0}, \mathbf{I})$ and transform them to samples from $\mathbf{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.

Using samples

$$\mathbf{x}^1 = (0.5, 1.1)^T, \mathbf{x}^2 = (0.1, 0.9)^T, \mathbf{x}^3 = (-0.2, 1.3)^T$$

and computing $\mathbf{y}^1 = \mathbf{A}\mathbf{x}^1 + \mathbf{t}$, we get:

$$\mathbf{y}^1 = (1.5, 3.2)^T, \mathbf{y}^2 = (1.1, 2.8)^T, \mathbf{y}^3 = (0.8, 3.6)^T.$$

Bonus question: Compute the sample covariance of the resulting data, and the actual covariance $\boldsymbol{\Sigma}$.

For the sample covariance, follow the same procedure as in the previous question. After a bit of calculating, we get:

$$\mathbf{S} \approx \begin{pmatrix} 0.123 & -0.06 \\ -0.06 & 0.16 \end{pmatrix}$$

The true covariance we get by computing $\Sigma = \mathbf{A}\mathbf{A}^T = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}$. They're quite different at the moment. If we increase the size of the sample, $\mathbf{S}$ will converge to Σ .